

Classical ML Algorithms: Detailed Placement and Interview Guide

This guide covers twelve foundational supervised-learning algorithms from first principles to interview depth. Unless stated otherwise, let $X \in \mathbb{R}^{n \times d}$ contain n examples and d features, y is the target, and a bias/intercept is fitted separately. Put preprocessing and the estimator in one scikit-learn **Pipeline** so transformations are learned only from training folds.

Placement strategy: Be able to explain the intuition, write the objective, state the important assumptions, discuss preprocessing and complexity, and justify the model/metric for a business problem. Memorizing an API is not enough.

Contents

1. [Linear Regression](#)
2. [Polynomial Regression](#)
3. [Ridge Regression](#)
4. [Lasso Regression](#)
5. [Elastic Net](#)
6. [Logistic Regression](#)
7. [Naive Bayes](#)
8. [K-Nearest Neighbors](#)
9. [Decision Tree](#)
10. [Random Forest](#)
11. [Extra Trees](#)
12. [Support Vector Machine](#)

Linear Regression

1. Overview

Linear regression predicts a continuous target as a weighted sum of input features. It is useful because it is fast, interpretable, statistically well understood, and a strong baseline. Real systems use it for price, demand, duration, risk, sensor calibration, and as an interpretable component inside larger pipelines.

2. Intuition

Fit the line or hyperplane that passes as close as possible to the observed points. For house prices, a coefficient of 2500 for area means that, holding other included variables fixed, one additional unit of area changes the predicted price by about 2500 units. This is an association unless the study design supports causality.

3. Prerequisites

- Vectors, matrices, transpose, inverse/pseudoinverse, dot product, rank.
- Mean, variance, covariance, residuals, probability distributions.
- Derivatives, gradients, convex optimization, train/test splitting.
- NumPy, pandas, scikit-learn, regression metrics.

4. Core Concepts

Concept	Meaning and why it matters	Simple example	Interview angle
Linear model	$\hat{y}$ is linear in parameters; features themselves may be transformed	$\hat{y} = 4 + 2x$	"Linear" refers to coefficients, not necessarily raw inputs
Coefficient	Partial change in prediction for a one-unit feature increase	rooms coefficient = 18,000	Interpretation depends on units and other variables
Intercept	Prediction when all features are zero	Base delivery time	May be meaningless if zero lies outside data range
Residual	$(e_i = y_i - \hat{y}_i)$	Actual 10, predicted 8, residual 2	Residual diagnostics reveal misspecification
Multicollinearity	Predictors contain overlapping information	area and number of rooms	Inflates coefficient variance; prediction may remain good
OLS	Choose coefficients minimizing squared residuals	Large errors are penalized strongly	Why MSE gives a closed-form convex problem

5. Algorithm / Working Process

1. Input numeric/categorical observations and a continuous target.
2. Split data before learning imputation, encoding, or scaling.
3. Build a design matrix; optionally add an intercept column.
4. Estimate coefficients by the normal equation, QR/SVD, or an iterative optimizer.
5. Output coefficients, intercept, and numeric predictions.
6. During inference, apply the exact training transformations and compute $(Xw+b)$.

6. Mathematical Foundation

Model and residual:

$$\hat{y}_i = w^T x_i + b, \quad e_i = y_i - \hat{y}_i.$$

Ordinary least squares (OLS) minimizes residual sum of squares; MSE differs only by a constant:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - w^T x_i - b)^2 = \frac{1}{n} \|y - Xw\|_2^2.$$

With a full-rank design matrix that already includes the intercept column:

$$\hat{w} = (X^T X)^{-1} X^T y.$$

In practice use QR, SVD, or $(X^T X + \lambda I)^{-1} X^T (Xw - y)$, not an explicit inverse. The gradient is $(\nabla_w J = \frac{2}{n} X^T (Xw - y))$. Under $(y = Xw + \epsilon)$, zero conditional error mean, independent observations, constant error variance, and no perfect multicollinearity, OLS has classical inferential guarantees. Normal residuals are needed for exact small-sample tests, not for fitting OLS itself. $(R^2 = 1 - \frac{\sum e_i^2}{\sum (y_i - \bar{y})^2})$.

7. Practical Implementation

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

X, y = fetch_california_housing(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = make_pipeline(SimpleImputer(strategy="median"), LinearRegression())
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred))
print("RMSE:", mean_squared_error(y_test, pred) ** 0.5)
print("R2:", r2_score(y_test, pred))
print("CV RMSE:", (-cross_val_score(
    model, X_train, y_train, scoring="neg_root_mean_squared_error", cv=5
)).mean())
```

8. Code Explanation

`train_test_split` preserves an untouched final estimate. The pipeline learns median values only from each training partition. `fit` estimates OLS coefficients; `predict` computes the linear response. MAE is easy to explain, RMSE emphasizes large misses, (R^2) compares against predicting the target mean, and cross-validation estimates variability across folds.

9. Training / Evaluation

Use a random split only for i.i.d. data; use time-based or grouped splitting when observations share time/entities. Inspect residual-versus-fitted plots for nonlinearity or changing variance and Q-Q plots only when inference assumptions matter. Track MAE/RMSE and (R^2) ; report errors in target units. Underfitting appears as systematic residual structure and poor train/test scores. Add useful features or nonlinear terms; use regularization when variance or multicollinearity is high.

10. Complexity and Cost

Dense least squares is roughly $(O(nd^2 + d^3))$ when $(n \geq d)$, with $(O(nd))$ data memory and $(O(d))$ per-example inference. Solver details change the exact cost. CPU is sufficient for ordinary tabular datasets; sparse or very large problems favor iterative SGD/LSQR solvers.

11. Common Use Cases

- Price, revenue, energy-load, and demand baselines.
- Marketing-mix and interpretable business-effect modeling.
- Calibration of sensors or laboratory instruments.
- Estimating time-to-complete or continuous risk scores.
- A baseline against which nonlinear models must earn their complexity.

12. Common Mistakes

- Treating association as causation or interpreting coefficients with omitted confounders.
- Fitting encoders/imputers before splitting; this leaks validation information.
- Ignoring nonlinear residual patterns, influential outliers, and heteroscedasticity.
- Comparing RMSE across targets with different units or using MAPE near zero.
- Explicitly calculating $((X^T X)^{-1})$, which is numerically fragile.
- Assuming feature scaling changes unregularized OLS predictions; it changes coefficient units, not the fitted space.

13. Edge Cases / Limitations

OLS extrapolates without bounds, is sensitive to high-leverage outliers, and cannot learn interactions/nonlinearity unless features encode them. Perfect collinearity makes coefficients non-identifiable. A high (R^2) can coexist with biased residuals, leakage, or useless causal interpretation. When $(d \gg n)$, infinitely many interpolating solutions may exist.

14. Variations

- **Weighted least squares:** weights observations; use for known unequal noise, important in statistics/interviews.
- **Robust regression (Huber/RANSAC):** reduces outlier influence; valuable in noisy projects.
- **Generalized least squares:** models correlated/nonconstant errors; statistics/research relevant.
- **Online SGD regression:** updates incrementally; useful for streaming or very large data.

15. Related Topics

Polynomial regression remains linear in expanded coefficients. Ridge stabilizes correlated coefficients; Lasso selects features. Logistic regression uses a linear score but models class log-odds. Generalized linear models replace the Gaussian response/link. PCA can address collinearity but sacrifices direct feature semantics.

16. Interview Questions

1. **Why is it called linear?** The prediction is linear in parameters; (x^2) may be a feature.
2. **Why square residuals?** It creates a differentiable convex objective and is the Gaussian-noise maximum-likelihood solution.
3. **What does a coefficient mean?** Expected prediction change per unit increase, holding included predictors fixed.
4. **When is OLS unbiased?** Most importantly when $(E[\epsilon \mid X] = 0)$ and the model is correctly specified.
5. **Does OLS require normal features?** No. Normal residuals matter for exact inference, not basic fitting.

6. **What does multicollinearity do?** Makes individual coefficients unstable and standard errors large.
7. **Can (R^2) be negative?** Yes on test data, or for a no-intercept model, when it loses to the mean baseline.
8. **Why not invert ($X^T X$) directly?** Poor conditioning and needless numerical error; QR/SVD are safer.
9. **MAE or RMSE?** MAE is robust/interpretable; RMSE penalizes large errors more.
10. **How do you detect heteroscedasticity?** Residual-vs-fitted plots or tests such as Breusch-Pagan; use robust errors/transformations if needed.

17. Practice Tasks

- Implement OLS with `np.linalg.lstsq`, then compare with scikit-learn.
- Predict California housing values and compare MAE/RMSE across feature sets.
- Add a synthetic outlier and analyze coefficient/residual changes.
- Diagnose leakage from scaling the full dataset before cross-validation.
- Extend the model with interactions and compare repeated-CV performance.

18. Project Ideas

Project	What it does	Stack and dataset	Resume value
House-price explainer	Predicts prices and explains coefficients/residuals	pandas, sklearn, SHAP; Ames Housing	End-to-end tabular modeling and diagnostics
Energy baseline	Forecasts hourly building usage	sklearn; UCI Appliances Energy	Time splits, lag features, business metrics
Delivery ETA	Predicts trip duration	FastAPI, sklearn; NYC Taxi sample	Feature pipeline plus deployable inference API

19. Quick Revision

- **Key idea:** best linear least-squares fit. **Formula:** $\hat{y} = (w^T x + b)$, minimize MSE.
- **Use:** continuous targets and interpretable baselines. **Metrics:** MAE, RMSE, (R^2).
- **Traps:** leakage, nonlinear residuals, outliers, causal claims.
- **Interview one-liner:** OLS projects (y) onto the column space of (X).

20. Final Cheat Sheet

Item	Summary
Definition	Linear-in-parameters continuous predictor
Input/output	Numeric feature matrix $\rightarrow$ real-valued prediction
Main steps	preprocess $\rightarrow$ solve least squares $\rightarrow$ diagnose residuals
Hyperparameters	Essentially none in basic OLS; <code>fit_intercept</code> , positivity constraint
Pros/cons	Fast, transparent / linear, outlier-sensitive, extrapolates
Best use	Interpretable regression and baseline modeling

Polynomial Regression

1. Overview

Polynomial regression expands original inputs into powers and interactions, then fits an ordinary or regularized linear model. It captures smooth nonlinear relationships while retaining linear-model training. It is used for calibration curves, small scientific datasets, response surfaces, and interpretable low-dimensional trends.

2. Intuition

A straight ruler cannot follow a curved road. Add (x^2, x^3) , and interaction features, then fit a straight model in that enlarged feature space. A parabola in the original plot is a hyperplane over $([x, x^2])$.

3. Prerequisites

Linear regression, algebraic powers/interactions, feature scaling, bias-variance trade-off, cross-validation, Ridge/Lasso basics.

4. Core Concepts

Concept	Meaning / example	Why and interview angle
Degree	Largest total exponent; degree 2 adds $(x_j^2, x_j x_k)$	Controls flexibility and variance
Basis expansion	Deterministic map $\phi(x)$ before a linear estimator	Model is nonlinear in inputs, linear in weights
Interaction	$(x_1 x_2)$: one feature's effect depends on another	<code>interaction_only=True</code> excludes pure powers
Feature explosion	Number grows as $\binom{d+p}{p}$ including bias	Major scalability and overfitting concern
Regularization	Ridge/Lasso restrains expanded coefficients	Usually safer for degree > 2

5. Algorithm / Working Process

Split data; learn ordinary preprocessing; map each row to polynomial features up to degree (p) ; scale expanded features when regularizing; fit coefficients; transform future rows identically; output continuous predictions. Choose degree and penalty only through training-fold cross-validation.

6. Mathematical Foundation

For one feature and degree (p) :

$$\hat{y} = b + w_1 x + w_2 x^2 + \dots + w_p x^p = w^T \phi(x) + b.$$

For two features at degree two, ϕ may contain $([x_1, x_2, x_1^2, x_1x_2, x_2^2])$. Training minimizes $(\|y - \Phi w\|_2^2/n)$, possibly plus a penalty. The model is still convex because Φ is fixed. Approximately $(d+p)\text{choose } p - 1$ output features are produced when the constant is excluded.

7. Practical Implementation

```
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(42)
X = rng.uniform(-3, 3, size=(500, 1))
y = 1.5 + 2 * X[:, 0] - 0.8 * X[:, 0] ** 2 + rng.normal(0, 1, 500)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

pipe = Pipeline([
    ("poly", PolynomialFeatures(include_bias=False)),
    ("scale", StandardScaler()),
    ("model", Ridge()),
])
search = GridSearchCV(
    pipe,
    {"poly__degree": [1, 2, 3, 5], "model__alpha": [0.01, 0.1, 1, 10]},
    scoring="neg_root_mean_squared_error",
    cv=5,
)
search.fit(X_train, y_train)
pred = search.predict(X_test)
print(search.best_params_)
print("Test RMSE:", mean_squared_error(y_test, pred) ** 0.5)
```

8. Code Explanation

The pipeline prevents polynomial expansion/scaling leakage. `include_bias=False` avoids duplicating the estimator intercept. Grid search selects both shape complexity and shrinkage using training folds. Ridge is used because polynomial columns are strongly correlated and high powers can create unstable coefficients.

9. Training / Evaluation

Plot train and validation error versus degree. Degree 1 may underfit; a large degree can nearly interpolate training data and fail outside it. Standardize after expansion, tune degree and regularization together, and use MAE/RMSE/ (R^2) . For time data use forward validation. Inspect predictions at domain boundaries because extrapolation can explode.

10. Complexity and Cost

If expansion produces ($D = \binom{d+p}{p} - 1$) columns, materialization costs ($O(nD)$) memory/time and dense fitting is roughly ($O(nD^2 + D^3)$). Inference is ($O(D)$). CPU is enough for small (d, p); the combinatorial expansion is the bottleneck, not GPU availability.

11. Common Use Cases

Calibration curves, physical response surfaces, dose-response trends, growth curves over a limited range, feature interactions in small tabular models.

12. Common Mistakes

Using a high degree without CV; expanding one-hot columns indiscriminately; scaling before the split; setting both a polynomial bias and model intercept; interpreting raw coefficients without accounting for scale/interactions; trusting far-range extrapolation.

13. Edge Cases / Limitations

Powers can overflow or become badly conditioned, high-dimensional expansion is combinatorial, global polynomials oscillate, and extrapolation is dangerous. A polynomial is a poor representation of discontinuities or complex local structure. Correlated powers make coefficients unstable without regularization.

14. Variations

- **Interaction-only models:** omit powers; useful for interpretable tabular interactions.
- **Orthogonal polynomials:** improve numerical conditioning; statistics/research.
- **Splines:** piecewise low-degree curves with controlled smoothness; often preferable for one-dimensional nonlinear trends.
- **Kernel polynomial methods:** compute implicit feature similarities; important for SVM/kernel interviews.

15. Related Topics

Compared with decision trees, polynomial models are smooth and global rather than piecewise constant. Splines offer local flexibility. Kernel methods can represent very large feature expansions implicitly. Ridge is the common stabilizer; Lasso can remove expanded terms but hierarchy is not automatically respected.

16. Interview Questions

1. **Is polynomial regression linear?** Linear in coefficients, nonlinear in original inputs.
2. **What does degree 2 add?** Squares plus pairwise products, unless interactions are disabled.
3. **Why scale?** High powers have vastly different magnitudes and penalties are scale-sensitive.
4. **Why overfit?** Feature count/flexibility rises rapidly with degree.
5. **How choose degree?** Cross-validation within the training set.
6. **Why Ridge?** Expanded columns are correlated; L2 stabilizes their weights.
7. **Why `include_bias=False`?** The downstream model already fits an intercept.
8. **How many terms?** ($\binom{d+p}{p}$) including the constant.
9. **Main extrapolation issue?** Highest-degree terms dominate outside the observed range.
10. **Polynomial versus spline?** Polynomial is global; splines provide controlled local curvature.

17. Practice Tasks

Implement a degree-2 expansion manually; reproduce it with `PolynomialFeatures`; plot learning curves for degrees 1/2/10; debug leakage caused by expansion before CV; compare polynomial Ridge with a shallow tree.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Sensor calibrator	Corrects nonlinear sensor readings	NumPy/sklearn; self-collected calibration	Modeling plus residual diagnostics
Yield response surface	Models fertilizer/weather interactions	pandas/sklearn; crop-yield dataset	Interactions and scientific explanation
Battery curve	Estimates capacity from cycle conditions	sklearn; NASA battery dataset	Domain-aware validation and extrapolation checks

19. Quick Revision

Expanded features + linear fitting; ($\hat{y} = w^T \phi(x)$); tune degree and penalty; use regression metrics; trap: combinatorial features and wild extrapolation. **One-liner:** polynomial regression is linear regression in a nonlinear basis.

20. Final Cheat Sheet

Item	Summary
Input/output	Features → expanded monomials → continuous value
Main steps	choose degree, expand, scale, regularize, validate
Hyperparameters	degree, interactions, Ridge/Lasso strength
Pros/cons	Smooth, interpretable / unstable, poor extrapolation
Best use	Low-dimensional smooth curvature and interactions

Ridge Regression

1. Overview

Ridge regression is linear regression with an L2 penalty on coefficients. It trades a little bias for lower variance, stabilizes estimates under multicollinearity, and works well when many features have small real effects. It is common in high-dimensional tabular data, text baselines, forecasting, and polynomial models.

2. Intuition

OLS may assign huge opposite weights to nearly duplicate features. Ridge attaches a spring to every weight, discouraging extreme values while allowing all features to contribute.

3. Prerequisites

OLS, vector norms, scaling, bias-variance trade-off, multicollinearity, cross-validation, eigenvalues/SVD at interview depth.

4. Core Concepts

Concept	Meaning / example	Interview angle
L2 penalty	Sum of squared weights	Shrinks but rarely makes weights exactly zero
α/λ	Regularization strength	0 approaches OLS; very large values approach an intercept-only fit
Scaling	Places coefficients under comparable penalties	Mandatory for meaningful regularization
Bias-variance	Adds bias, often reduces test error variance	Why training error rises while validation improves
Multicollinearity	$(X^T X + \lambda I)$ becomes better conditioned	Core reason Ridge is stable

5. Algorithm / Working Process

Split, impute/encode, standardize features, choose candidate α values, fit penalized least squares in each training fold, select α by a validation metric, refit, and predict $(Xw+b)$. The intercept is normally not penalized.

6. Mathematical Foundation

$$\min_{w,b} \frac{1}{n} \|y - Xw - b\mathbf{1}\|_2^2 + \lambda \|w\|_2^2.$$

With centered data, a common closed form is

$$\hat{w} = (X^T X + n\lambda I)^{-1} X^T y,$$

where the factor (n) depends on the library's objective convention. In the SVD basis ($X=U\Sigma V^T$), Ridge scales component (j) by $(\sigma_j/(\sigma_j^2 + \lambda))$, suppressing poorly identified directions with small singular values. Its constraint form is $(\|w\|_2^2 \leq t)$.

7. Practical Implementation

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV
```

```

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

X, y = fetch_california_housing(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
model = make_pipeline(
    StandardScaler(),
    RidgeCV(alphas=np.logspace(-4, 4, 25), cv=5,
            scoring="neg_root_mean_squared_error"),
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
ridge = model.named_steps["ridgecv"]
print("alpha:", ridge.alpha_)
print("MAE/RMSE/R2:", mean_absolute_error(y_test, pred),
      mean_squared_error(y_test, pred) ** 0.5, r2_score(y_test, pred))

```

8. Code Explanation

Scaling and Ridge live in one pipeline, so each CV fold learns its own scale. `np.logspace` searches orders of magnitude because useful α values are rarely known linearly. `RidgeCV` selects α using CV RMSE and then refits on the training set.

9. Training / Evaluation

Compare against OLS using identical folds. Tune α on a log scale and examine coefficient paths. If both train and validation error are high, regularization may be excessive or features inadequate. If the gap is large, increase α or acquire more data. Use MAE/RMSE/(R^2); use grouped/time splits when applicable.

10. Complexity and Cost

Depending on solver and shape, dense training is about ($O(nd^2 + d^3)$) or iterative ($O(knd)$); inference is ($O(d)$); model memory is ($O(d)$). Multiple α /CV fits multiply training cost. CPU handles most use cases; sparse solvers suit high-dimensional text.

11. Common Use Cases

Correlated economic predictors, polynomial features, text regression, genomics with many weak signals, stable baseline models, linear models where feature selection is not required.

12. Common Mistakes

Not scaling; tuning α on test data; claiming Ridge performs feature selection; comparing α across libraries without checking objective conventions; penalizing encoded features inconsistently; interpreting shrunk coefficients as unbiased effects.

13. Edge Cases / Limitations

Ridge keeps irrelevant variables, remains linear, is outlier-sensitive under squared loss, and may underfit when α is too large. It does not solve leakage, confounding, or nonlinear misspecification. With sparse inputs, centering can destroy sparsity, so use compatible scalers/solvers.

14. Variations

- **Kernel Ridge:** nonlinear prediction via kernels; useful for medium-size smooth problems.
- **Bayesian Ridge:** probabilistic priors and uncertainty estimates; research/statistics.
- **Tikhonov regularization:** generalized penalty ($\|Lw\|^2$); structured inverse problems.
- **Ridge classifier:** least-squares-style linear classification; fast baseline.

15. Related Topics

Lasso uses L1 and creates sparsity; Elastic Net mixes L1/L2. PCA regression removes low-variance directions rather than continuously shrinking them. Weight decay is the neural-network optimization analogue of L2 under common conditions.

16. Interview Questions

1. **Why Ridge?** Lower variance and better conditioning for correlated/high-dimensional features.
2. **Does it select features?** Usually no; L2 produces small but nonzero coefficients.
3. **Why standardize?** Otherwise large-unit features receive effectively different penalties.
4. **What happens at $\alpha=0$?** It becomes OLS when the solution is well defined.
5. **At $\alpha \rightarrow \infty$?** Slopes approach zero; the intercept remains.
6. **Why does it handle multicollinearity?** Adds positive mass to eigenvalues of $(X^T X)$.
7. **Is the intercept penalized?** Conventionally no.
8. **How choose α ?** Cross-validation on a log-spaced grid.
9. **Ridge versus PCA?** Ridge softly shrinks all singular directions; PCA truncates selected directions.
10. **Can Ridge outperform OLS with some bias?** Yes, reduced variance can lower expected test error.

17. Practice Tasks

Plot coefficient paths versus α ; simulate two correlated predictors; compare OLS condition number and Ridge stability; debug unscaled Ridge; extend the model with polynomial features.

18. Project Ideas

Project	What/stack/dataset	Resume value
Salary predictor	sklearn + Stack Overflow survey	Handles correlated encoded features
News engagement	TF-IDF + Ridge; news popularity data	Sparse high-dimensional modeling
Demand baseline	Ridge with lag/calendar features; store sales	Time validation and stable deployment

19. Quick Revision

OLS + $(\lambda \|w\|_2^2)$; use for correlated/many weak features; tune α with CV; scale first; no exact sparsity.

One-liner: Ridge buys stability by shrinking coefficients in poorly determined directions.

20. Final Cheat Sheet

Item	Summary
Input/output	Scaled features → continuous value
Key hyperparameter	α/λ regularization strength
Metrics	MAE, RMSE, (R^2), CV stability
Pros/cons	Stable, convex / dense weights, linear, outlier-sensitive
Best use	Multicollinearity and many weak predictors

Lasso Regression

1. Overview

Lasso (Least Absolute Shrinkage and Selection Operator) adds an L1 coefficient penalty. Its geometric corners can drive coefficients exactly to zero, combining regression and embedded feature selection. It is useful when a sparse subset of features is expected.

2. Intuition

Imagine a strict total budget for absolute coefficient sizes. The model spends that budget on the most useful variables and may give the rest exactly zero.

3. Prerequisites

Linear regression, L1/L2 norms, scaling, convex but nondifferentiable optimization, sparsity, coordinate descent, cross-validation.

4. Core Concepts

Concept	Meaning	Why/interview angle
L1 penalty	$(\sum_j$	w_j
Sparsity	Some coefficients exactly zero	Embedded feature selection
Soft thresholding	Shrink toward zero, then clip small values to zero	Coordinate-descent update intuition
Correlated predictors	Lasso may arbitrarily choose one	Selection can be unstable
Convergence	Iterative solver stops by tolerance/duality gap	Scale features and raise iterations if warned

5. Algorithm / Working Process

Split and preprocess, standardize numeric features, initialize weights, repeatedly optimize one coefficient while holding others fixed (soft threshold), stop at tolerance, choose α by CV, refit, and predict linearly. Zero

coefficients identify features selected for this fitted sample and penalty.

6. Mathematical Foundation

$$\min_{w,b} \frac{1}{2n} \|y - Xw - b\mathbf{1}\|_2^2 + \lambda \|w\|_1.$$

For an orthonormal design, ($w_j = S_\lambda(w_j^{\text{OLS}})$), with soft-threshold operator

$$S_\lambda(z) = \text{sign}(z) \max(|z| - \lambda, 0).$$

L1 is convex but not differentiable at zero; subgradients or coordinate descent solve it. The equivalent constraint is ($\|w\|_1 \leq t$). Larger λ means more shrinkage and usually fewer selected features.

7. Practical Implementation

```
import numpy as np
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LassoCV
from sklearn.metrics import mean_squared_error, r2_score

X, y, true_w = make_regression(
    n_samples=800, n_features=100, n_informative=10,
    noise=15, coef=True, random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
model = make_pipeline(
    StandardScaler(),
    LassoCV(alphas=np.logspace(-3, 2, 60), cv=5,
            max_iter=20_000, random_state=42),
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
lasso = model.named_steps["lassocv"]
print("alpha/nonzero:", lasso.alpha_, np.count_nonzero(lasso.coef_))
print("RMSE/R2:", mean_squared_error(y_test, pred) ** 0.5,
      r2_score(y_test, pred))
```

8. Code Explanation

The synthetic target has only ten informative features, making selection inspectable. Scaling gives coefficients comparable L1 treatment. `LassoCV` chooses α without using the test set. `max_iter` is raised because coordinate descent can need more iterations at weak penalties or with correlated columns.

9. Training / Evaluation

Report predictive error and selection stability across resamples; a single nonzero set is not scientific proof of relevance. Tune α through nested CV if an unbiased tuned-model estimate is important. Underfitting appears as too few features/high train error; overfitting as weak α and unstable selections. Check `n_iter_/convergence` warnings.

10. Complexity and Cost

Coordinate descent is roughly $O(knd)$, with iteration count (k) sensitive to conditioning and tolerance. CV multiplies fits. Inference is $O(s)$ if sparse weights with $(s \ll d)$; parameter memory is $O(d)$. CPU is usually ideal.

11. Common Use Cases

High-dimensional biomarker screening, sparse economic models, text/count regression, compact feature pipelines, interpretable baselines where most candidate variables are believed irrelevant.

12. Common Mistakes

Skipping scaling; using zero coefficients as causal proof; ignoring unstable choices among correlated variables; tuning on test data; silently accepting non-convergence; one-hot encoding with a penalty that ignores group semantics; expecting good performance when the truth is dense.

13. Edge Cases / Limitations

When $(d > n)$, basic Lasso selects at most about (n) variables under general position. With strongly correlated predictors it can choose one unpredictably. It is biased for large true coefficients, remains linear, and squared loss remains sensitive to response outliers.

14. Variations

- **LassoLars**: follows a coefficient path efficiently in some low-sample settings.
- **Adaptive Lasso**: feature-specific weights can improve selection consistency; research/statistics.
- **Group Lasso**: selects predefined groups together; projects with grouped one-hot or sensors.
- **MultiTaskLasso**: selects shared features across several targets.

15. Related Topics

Ridge retains correlated groups; Elastic Net often selects correlated groups more stably. Recursive feature elimination is wrapper selection and costs repeated model fits. Compressed sensing provides theoretical sparse-recovery connections.

16. Interview Questions

1. **Why zeros?** L1's nondifferentiable corner makes zero an optimum for weak features.
2. **Lasso versus Ridge?** Sparse L1 selection versus dense L2 shrinkage.
3. **Why scale?** Penalty acts on coefficient magnitude, which depends on feature units.
4. **What if features correlate?** Lasso may select one arbitrarily and be unstable.
5. **How tune α ?** Cross-validation, usually over logarithmic values.
6. **What does larger α do?** More shrinkage, fewer nonzero weights, more bias.

7. **Is selection causal?** No; it is predictive and sample-dependent.
8. **Why convergence warnings?** Poor scaling, tight tolerance, low α , or correlated/high-dimensional data.
9. **Can Lasso select more than (n) variables when (d>n)?** Standard solutions generally cannot under general-position conditions.
10. **When prefer Elastic Net?** When sparsity is useful but predictors form correlated groups.

17. Practice Tasks

Implement soft thresholding; plot a regularization path; measure Jaccard stability of selected features over bootstraps; fix a convergence warning; compare Lasso, Ridge, and Elastic Net on sparse versus dense truth.

18. Project Ideas

Project	What/stack/dataset	Resume value
Gene-expression selector	sklearn; public microarray data	High-dimensional selection discipline
Marketing driver model	pandas/sklearn; advertising data	Sparse explainable business model
Compact text scorer	TF-IDF + Lasso; review helpfulness	Sparse NLP and deployment footprint

19. Quick Revision

MSE + $(\lambda|w|_1)$; use for sparse signal; tune α ; evaluate prediction and selection stability; trap: correlated predictors. **One-liner:** Lasso is a convex linear model that shrinks and selects through an L1 penalty.

20. Final Cheat Sheet

Item	Summary
Input/output	Scaled features → continuous prediction + sparse coefficients
Hyperparameters	α , tolerance, max iterations, positive constraint
Pros/cons	Embedded selection / biased, unstable with correlation
Metrics	RMSE/MAE/(R^2); selection stability when relevant
Best use	Many features with plausibly sparse truth

Elastic Net

1. Overview

Elastic Net combines L1 and L2 penalties. It can produce sparse models like Lasso while stabilizing and grouping correlated predictors like Ridge. It is often the safest regularized linear option for high-dimensional tabular, omics, and sparse text data.

2. Intuition

L1 imposes a limited coefficient budget; L2 places a spring on every weight. Together, unhelpful features can be removed while related useful features can share credit instead of one being chosen arbitrarily.

3. Prerequisites

OLS, Ridge, Lasso, norms, scaling, coordinate descent, cross-validation, correlated predictors.

4. Core Concepts

Concept	Meaning	Interview angle
Overall α	Total regularization	Larger means stronger shrinkage
<code>l1_ratio</code> ρ	Mix: 1=Lasso, 0=Ridge-like	Tune jointly with α
Grouping effect	Correlated predictors tend to enter/leave together	Advantage over pure Lasso
Sparsity	L1 component produces exact zeros	Vanishes as mix approaches pure L2
Scaling	Equalizes penalty across units	Must be inside CV pipeline

5. Algorithm / Working Process

Preprocess and standardize in the pipeline; define grids for α and L1 ratio; use coordinate descent to minimize the combined objective in each fold; select by validation error; refit; output predictions and coefficients.

6. Mathematical Foundation

A common parameterization is

$$\min_{w,b} \frac{1}{2n} \|y - Xw - b\mathbf{1}\|_2^2 + \alpha \rho \|w\|_1 + \frac{\alpha(1-\rho)}{2} \|w\|_2^2.$$

$\rho=1$ gives Lasso; ρ near 0 is mostly Ridge, though scikit-learn's `ElasticNet` is not the preferred implementation for exactly $\rho=0$. The L2 term makes correlated/underdetermined problems more stable, while the L1 subgradient creates zeros.

7. Practical Implementation

```
import numpy as np
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import ElasticNetCV
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=700, n_features=80, n_informative=15,
                      effective_rank=30, noise=12, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
```

```

model = make_pipeline(
    StandardScaler(),
    ElasticNetCV(l1_ratio=[0.1, 0.5, 0.8, 0.95, 1.0],
                 alphas=np.logspace(-3, 2, 50), cv=5,
                 max_iter=20_000, random_state=42),
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
enet = model.named_steps["elasticnetcv"]
print("alpha/l1_ratio:", enet.alpha_, enet.l1_ratio_)
print("nonzero/RMSE:", np.count_nonzero(enet.coef_),
      mean_squared_error(y_test, pred) ** 0.5)

```

8. Code Explanation

`effective_rank` creates correlated structure. The pipeline scales within validation folds. CV searches both how much total shrinkage to apply and how sparse versus stable it should be. Nonzero count is diagnostic, not the primary selection criterion.

9. Training / Evaluation

Tune α and p together using MAE/RMSE. Compare with Ridge and Lasso on identical folds. Use nested CV for unbiased model-selection evaluation. Analyze coefficient paths and stability. Increase iterations or loosen tolerance only after ensuring scaling. A very small L1 component may require a more careful α search.

10. Complexity and Cost

Coordinate descent is roughly ($O(knd)$) per fit; grid/CV cost scales with ratios, alphas, and folds. Warm starts along an α path reduce work. Inference is ($O(s)$) with (s) nonzero features; CPU is sufficient for typical datasets.

11. Common Use Cases

Genomics with correlated genes, financial/economic indicators, TF-IDF regression, sensor banks, polynomial/interacting features, and any high-dimensional problem where sparse but grouped signal is plausible.

12. Common Mistakes

Not scaling; tuning only α while assuming a mix; confusing scikit-learn's `l1_ratio` with other libraries' parameterization; treating selected features as causal; testing on the validation-tuned test set; accepting non-convergence.

13. Edge Cases / Limitations

It adds another hyperparameter, coefficients remain biased, group selection is an effect rather than a formal group constraint, and it cannot discover nonlinearities without basis features. Pure Ridge may be better for dense truth; pure Lasso may be easier when predictors are weakly correlated.

14. Variations

- **MultiTaskElasticNet:** shared sparsity across continuous targets; multi-output projects.
- **Sparse-group Lasso/Elastic Net:** selects groups and members; structured research.
- **Logistic Elastic Net:** same penalties with log loss; classification placements/projects.
- **Adaptive Elastic Net:** data-dependent weights; research-level selection consistency.

15. Related Topics

Elastic Net interpolates Ridge and Lasso. Group Lasso encodes explicit groups; PCA changes the representation instead. Logistic regression can use the same penalty family. Regularized Cox models extend the idea to survival analysis.

16. Interview Questions

1. **Why combine penalties?** Obtain sparsity without Lasso's full instability on correlated features.
2. **What is `l1_ratio=1`?** Lasso.
3. **What does low `l1_ratio` imply?** Mostly L2 shrinkage and a denser solution.
4. **Why tune both parameters?** Total strength and penalty shape affect different behavior.
5. **Why scale?** L1/L2 operate on coefficient magnitude and therefore feature units.
6. **What is grouping effect?** Strongly correlated predictors can receive similar nonzero weights.
7. **Does it guarantee semantic groups?** No; use group penalties for explicit group selection.
8. **When choose Ridge?** Dense signal where selection is unnecessary.
9. **When choose Lasso?** Sparse, weakly correlated signal and maximum simplicity.
10. **Main optimization method?** Coordinate descent for common squared-loss implementations.

17. Practice Tasks

Draw validation heatmaps over α/ρ ; duplicate noisy correlated features and compare Lasso; measure selection stability; diagnose an unscaled pipeline; extend to multi-output regression.

18. Project Ideas

Project	What/stack/dataset	Resume value
Biomarker panel	sklearn; gene-expression dataset	Correlated sparse selection
Portfolio factor model	pandas/sklearn; public market factors	Time-aware regularized modeling
Article engagement	TF-IDF + Elastic Net; news data	Sparse NLP regression and interpretation

19. Quick Revision

MSE + mixed L1/L2; tune α and `l1_ratio`; use for correlated sparse features; evaluate CV RMSE and stability; trap: mismatched parameter conventions. **One-liner:** Elastic Net is Lasso made less brittle by Ridge.

20. Final Cheat Sheet

Item	Summary
Definition	Linear regression with combined L1/L2 penalty

Item	Summary
Input/output	Scaled features → continuous prediction, partly sparse weights
Hyperparameters	α , <code>l1_ratio</code> , tolerance, iterations
Pros/cons	Sparse + stable / extra tuning, biased, linear
Best use	High-dimensional correlated predictors

Logistic Regression

1. Overview

Logistic regression is a discriminative linear classifier that maps a linear score to a class probability. Despite its name, it performs classification. It is fast, interpretable, naturally probabilistic, and widely used for credit risk, churn, conversion, medical screening, spam baselines, and high-dimensional sparse text.

2. Intuition

First compute evidence ($z = w^T x + b$). The sigmoid bends any real score into $(0,1)$. A decision threshold converts that probability into a label. Each coefficient adds a fixed amount to log-odds, so evidence combines additively.

3. Prerequisites

Linear models, probability/odds/log-odds, Bernoulli distribution, maximum likelihood, cross-entropy, gradients, regularization, classification metrics.

4. Core Concepts

Concept	Meaning / example	Why and interview angle
Logit	$\log(p/(1-p)) = w^T x + b$	Coefficients are linear in log-odds
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$	Produces a binary probability
Threshold	Label 1 if probability $\geq t$	0.5 is not universally optimal
Log loss	Penalizes confident wrong probabilities	Maximum-likelihood objective
Decision boundary	$(w^T x + b = 0)$ at $t=0.5$	Linear in feature space
Regularization	L2/L1/Elastic Net controls weights	<code>C</code> is inverse strength in sklearn

5. Algorithm / Working Process

Input features and categorical labels; split with stratification/group/time logic; preprocess and scale; initialize weights; compute probabilities; minimize regularized log loss iteratively; output probabilities and thresholded classes. At inference, use the fixed transformer, compute score/sigmoid (or softmax), then apply a business-selected threshold.

6. Mathematical Foundation

$$p_i = P(y_i = 1 \mid x_i) = \sigma(w^T x_i + b).$$

Bernoulli likelihood and negative log-likelihood:

$$L = \prod_i p_i^{y_i} (1 - p_i)^{1-y_i}, \quad J = -\frac{1}{n} \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)] + \lambda R(w).$$

The gradient without penalty is $(X^T(p-y))/n$. There is no ordinary closed form; solvers use Newton, quasi-Newton, SAG/SAGA, or coordinate-descent-like methods. For coefficient (w_j) , a one-unit feature increase multiplies odds by $(e^{\{w_j\}})$, holding other variables fixed. Multiclass softmax uses $(P(y=k \mid x) = e^{\{w_k^T x\}} / \sum_c e^{\{w_c^T x\}})$.

7. Practical Implementation

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score,
average_precision_score

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
pipe = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=5_000, class_weight="balanced",
solver="liblinear"),
)
search = GridSearchCV(pipe, {"logisticregression__C": [0.01, 0.1, 1, 10],
                           "logisticregression__penalty": ["l1", "l2"]},
                       scoring="roc_auc", cv=5)
search.fit(X_train, y_train)
prob = search.predict_proba(X_test)[: , 1]
pred = (prob >= 0.40).astype(int) # chosen from validation/business cost, not
test
print(search.best_params_)
print(classification_report(y_test, pred))
print("ROC-AUC/AP:", roc_auc_score(y_test, prob),
      average_precision_score(y_test, prob))
```

8. Code Explanation

Stratification preserves class ratios. Scaling improves solver conditioning and makes penalties comparable. `class_weight="balanced"` changes loss weights, not the evaluation distribution. Grid search chooses

regularization without the test set. Probabilities are separated from decisions so threshold choice can reflect false-positive/false-negative cost.

9. Training / Evaluation

For imbalance, report precision, recall, F1, PR-AUC, confusion matrix, and ROC-AUC—not accuracy alone. Evaluate probability quality with log loss, Brier score, and calibration plots. Choose a threshold on validation data under an explicit cost/capacity constraint. Underfitting: poor train and validation results; add features/interactions or reduce regularization. Overfitting: gap/high coefficient magnitude; strengthen penalty or get data.

10. Complexity and Cost

Each first-order iteration costs about $O(nd)$; total is $O(knd)$, solver-dependent. Model memory and per-row inference are $O(d)$. Multiclass softmax stores $O(cd)$. Linear logistic regression is CPU-friendly and scales to sparse high-dimensional inputs with suitable solvers.

11. Common Use Cases

Credit default, churn, conversion propensity, disease risk, fraud triage, moderation/spam, click-through probability, and interpretable binary baselines.

12. Common Mistakes

Calling outputs calibrated merely because they are probabilities; using accuracy on imbalance; scaling before splitting; interpreting coefficients without feature units; confusing C with λ (C is inverse strength); choosing a threshold on test data; ignoring perfect/quasi separation and convergence warnings.

13. Edge Cases / Limitations

Raw decision boundaries are linear; interactions must be engineered or kernelized. Severe separation can make unregularized MLE diverge. Correlation destabilizes coefficients. Rare classes yield uncertain probability estimates. Distribution shift invalidates calibration and thresholds even if rank metrics remain acceptable.

14. Variations

- **Multinomial logistic/softmax:** joint multiclass probabilities; placement essential.
- **One-vs-rest:** one binary classifier per class; useful with sparse/multilabel data.
- **L1/Elastic-Net logistic:** feature selection/group stability; projects.
- **Ordinal logistic:** ordered categories; statistics/product surveys.
- **Firth logistic:** reduces small-sample/separation bias; research/biostatistics.

15. Related Topics

Versus linear SVM, logistic regression optimizes log loss and produces probabilities; SVM optimizes margin. Naive Bayes is generative and assumes conditional independence. Linear regression predicts unbounded values. Calibration can improve SVM/tree probability estimates. Shallow neural networks generalize the linear-score-plus-link idea.

16. Interview Questions

1. **Why “regression”?** It models continuous log-odds, then classifies.
2. **Why sigmoid?** It maps a real score to a differentiable probability and yields linear log-odds.
3. **What is the loss?** Bernoulli negative log-likelihood/cross-entropy.
4. **Coefficient interpretation?** (e^{w_j}) is the odds multiplier for one unit, other variables fixed.
5. **Why no closed form?** The nonlinear likelihood equations cannot generally be isolated algebraically.
6. **What is C ?** Inverse regularization strength; smaller C means stronger shrinkage.
7. **Why not always threshold 0.5?** Costs, prevalence, and capacity differ.
8. **Logistic versus SVM?** Likelihood/probability versus maximum margin; both are linear unless transformed/kernelized.
9. **What is separation?** A feature combination perfectly splits classes, causing unregularized coefficients to diverge.
10. **How handle imbalance?** Appropriate split/metrics, weights or resampling inside folds, threshold tuning, and calibration checks.

17. Practice Tasks

Implement sigmoid/log loss/gradient descent; plot precision-recall versus threshold; compare class weighting with resampling; debug leakage from global scaling; add polynomial interactions and assess calibration.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Credit-risk scorecard	Predicts default with reason codes	pandas/sklearn; UCI Default	Imbalance, calibration, explainability
Churn intervention	Ranks accounts and selects threshold by budget	sklearn/FastAPI; Telco Churn	Business-aware decisions
Toxic-text baseline	TF-IDF logistic classifier	sklearn; Jigsaw subset	Sparse NLP, multilabel metrics

19. Quick Revision

Linear log-odds, sigmoid probability, log loss; scale/tune C ; metrics depend on imbalance and cost; trap: 0.5 threshold and probability $\neq$ calibration. **One-liner:** logistic regression is a regularized linear model trained by Bernoulli maximum likelihood.

20. Final Cheat Sheet

Item	Summary
Input/output	Features $\rightarrow$ class probability and label
Main steps	linear score $\rightarrow$ sigmoid/softmax $\rightarrow$ threshold/argmax
Hyperparameters	C , penalty, solver, class weights, threshold
Metrics	log loss, ROC-AUC, PR-AUC, F1, recall, calibration

Item	Summary
Pros/cons	Fast, interpretable / linear boundary, sensitive to specification
Best use	Probabilistic linear baseline and sparse data

Naive Bayes

1. Overview

Naive Bayes is a family of generative probabilistic classifiers applying Bayes' theorem with conditional independence of features given the class. It trains extremely quickly, needs relatively little data, supports incremental fitting in several variants, and excels as a sparse text baseline.

2. Intuition

For each class, learn how common the class is and how likely each observed clue is under that class. Multiply the clues with the prior and select the most plausible class. In spam filtering, "free" and "winner" each add evidence even though the model naively treats them as independent after conditioning on spam/non-spam.

3. Prerequisites

Conditional probability, Bayes' theorem, likelihood/prior/posterior, logarithms, basic distributions, count/TF-IDF features, classification metrics.

4. Core Concepts

Concept	Meaning	Example/interview angle
Class prior	$P(y=c)$	Class frequency or domain prior
Likelihood	$P(x_j y=c)$	Gaussian density or token frequency
Naive assumption	Features independent conditional on class	False often, yet classification can work
MAP rule	Choose largest posterior	Evidence denominator cancels across classes
Smoothing	Adds pseudo-count α	Avoids zeroing entire product for unseen words
Log-space	Sum log probabilities	Prevents floating-point underflow

5. Algorithm / Working Process

Estimate class priors; estimate class-conditional feature-distribution parameters; for a new sample compute each class's log prior plus summed log likelihoods; select the maximum score; normalize if probabilities are requested. Training estimates counts/means/variances rather than gradient-descent weights.

6. Mathematical Foundation

$$P(y = c \mid x) = \frac{P(y = c)P(x \mid y = c)}{P(x)} \propto P(y = c) \prod_{j=1}^d P(x_j \mid y = c).$$

Decision rule:

$$\hat{y} = \arg \max_c \left[\log P(c) + \sum_j \log P(x_j \mid c) \right].$$

For Multinomial NB token probability with smoothing:

$$\hat{\theta}_{cj} = \frac{N_{cj} + \alpha}{N_c + \alpha d}.$$

Gaussian NB assumes $(x_j \mid y=c) \sim \mathcal{N}(\mu_{\{c\}}, \sigma_{\{c\}}^2)$. Bernoulli NB models binary presence. The posterior probabilities may be poorly calibrated because dependence is counted repeatedly.

7. Practical Implementation

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, confusion_matrix

docs, y = fetch_20newsgroups(subset="train", categories=[
    "sci.space", "rec.sport.baseball", "comp.graphics", "talk.politics.misc"
], remove=("headers", "footers", "quotes"), return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    docs, y, test_size=0.2, stratify=y, random_state=42
)
model = make_pipeline(
    TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True),
    MultinomialNB(alpha=0.5),
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(classification_report(y_test, pred))
print(confusion_matrix(y_test, pred))
```

8. Code Explanation

Text stays raw until the pipeline, preventing vocabulary/IDF leakage. TF-IDF values are nonnegative, satisfying Multinomial NB's feature requirement even though they are not literal counts. `alpha` smooths unseen terms. Macro metrics expose performance across all four classes.

9. Training / Evaluation

Match distribution to representation: Gaussian for continuous roughly class-normal features, Multinomial/Complement for nonnegative counts, Bernoulli for presence, Categorical for encoded categories. Tune α and text features inside CV. Use macro-F1/PR metrics for imbalance. Validate probabilities with log loss/calibration before cost decisions. Learning curves show NB's small-data advantage.

10. Complexity and Cost

Training and inference are roughly $O(nd)$ and $O(cd)$ per dense example, or proportional to nonzeros for sparse text. Parameters require $O(cd)$. It is CPU-friendly, supports very large vocabularies, and several variants provide `partial_fit` for out-of-core data.

11. Common Use Cases

Spam, sentiment/topic/document classification, intent routing, language identification, medical diagnosis baselines, real-time incremental classification.

12. Common Mistakes

Using Gaussian NB for word counts; feeding negative values to Multinomial NB; vectorizing before splitting; omitting smoothing; multiplying raw probabilities and underflowing; trusting `predict_proba` as calibrated; interpreting independence as marginal rather than class-conditional.

13. Edge Cases / Limitations

Highly redundant correlated evidence creates overconfident scores. Gaussian density estimates can fail with skew/multimodal features or near-zero variance. Unseen categories/tokens require smoothing and consistent encoding. The boundary imposed by the distribution may be too simple.

14. Variations

- **GaussianNB:** continuous class-conditional Gaussian features; standard interview variant.
- **MultinomialNB:** counts/nonnegative frequencies; essential for NLP.
- **BernoulliNB:** binary feature presence/absence; short text.
- **ComplementNB:** statistics from complementary classes; often stronger for imbalanced text.
- **CategoricalNB:** categorical-distributed encoded features; tabular categories.

15. Related Topics

Naive Bayes models ($P(x,y)$); logistic regression models ($P(y \mid x)$). Linear discriminant analysis permits correlated features through covariance assumptions. Bag-of-words representations make NB especially natural. Calibration methods may repair probability estimates but require held-out data.

16. Interview Questions

1. **What is naive?** Conditional feature independence given the class.
2. **Why can it work when the assumption is false?** Correct class ranking can survive inaccurate probability estimates.
3. **Why log probabilities?** Products of many small values underflow; logs turn products into sums.
4. **Why smoothing?** An unseen feature-class pair would otherwise make the full likelihood zero.

5. **Gaussian versus Multinomial?** Continuous Gaussian density versus nonnegative count/frequency model.
6. **What is learned?** Priors and class-conditional distribution parameters.
7. **Is it generative?** Yes; it models class priors and $P(x \mid y)$.
8. **Why fast?** Independent sufficient statistics; no iterative joint optimization.
9. **Are its probabilities reliable?** Often overconfident; evaluate/calibrate them.
10. **NB versus logistic for text?** NB often wins with little data; logistic typically improves with enough labeled data and correlated evidence.

17. Practice Tasks

Implement Multinomial NB in log-space; compare count versus TF-IDF; tune smoothing; construct a zero-frequency failure; compare calibration of NB and logistic regression.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Spam gateway	Classifies and explains suspicious tokens	sklearn/FastAPI; SMS Spam	Sparse pipeline and low-latency API
News router	Routes articles to topic queues	sklearn; 20 Newsgroups	Multiclass error analysis
Incremental ticket triage	Updates from streaming batches	<code>partial_fit</code> ; support-ticket data	Online learning and monitoring

19. Quick Revision

Prior $\times$ independent likelihoods, compute in log-space; select distribution carefully; tune smoothing; metrics by class; trap: leakage and uncalibrated probabilities. **One-liner:** Naive Bayes is a fast generative classifier that turns conditionally independent feature evidence into a MAP decision.

20. Final Cheat Sheet

Item	Summary
Input/output	Distribution-compatible features $\rightarrow$ posterior-like scores/class
Main steps	estimate priors/likelihoods $\rightarrow$ sum log evidence $\rightarrow$ argmax
Hyperparameters	smoothing α , class priors, variance smoothing
Metrics	macro-F1, PR-AUC, log loss/calibration if using probabilities
Pros/cons	Very fast, small-data / independence, probability quality
Best use	Sparse text and cheap probabilistic baseline

K-Nearest Neighbors

1. Overview

K-Nearest Neighbors (KNN) is a nonparametric, instance-based method for classification and regression. It stores training examples and predicts from the nearby examples under a distance metric. It works well for small, low-dimensional datasets with irregular local boundaries.

2. Intuition

To judge a new neighborhood, ask the most similar nearby neighborhoods. Their majority class or average target becomes the prediction. Small (k) listens to a few possibly noisy neighbors; large (k) smooths across a wider region.

3. Prerequisites

Distance metrics, feature scaling, majority vote/averaging, bias-variance, curse of dimensionality, train-validation splitting, KD/Ball trees.

4. Core Concepts

Concept	Meaning	Example/interview angle
(k)	Number of neighbors	Odd (k) reduces binary vote ties but does not guarantee none
Metric	Definition of closeness	Euclidean, Manhattan, Minkowski, cosine (via suitable setup)
Weighting	Uniform or inverse-distance votes	Nearby samples may deserve more influence
Lazy learning	Little parametric fitting; store data	Cheap training, expensive inference
Scaling	Distance is unit-sensitive	Income otherwise dominates age
Curse of dimensionality	Distances become less discriminative	Feature selection/PCA may help

5. Algorithm / Working Process

Preprocess and scale training data; retain examples or build a search index; for each query calculate/search distances; identify (k) smallest; aggregate labels by vote/probability or targets by mean; return prediction. There is no learned global decision function.

6. Mathematical Foundation

Minkowski distance:

$$d_p(x, z) = \left(\sum_{j=1}^d |x_j - z_j|^p \right)^{1/p}.$$

(p=2) is Euclidean; (p=1) Manhattan. Classification uses

$$\hat{y} = \arg \max_c \sum_{i \in N_k(x)} w_i \mathbf{1}(y_i = c),$$

and regression uses $(\hat{y} = \sum w_i y_i / \sum w_i)$, often $(w_i = 1/(d(x, x_i) + \epsilon))$. Small (k) has low bias/high variance; large (k) has higher bias/lower variance.

7. Practical Implementation

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)
pipe = make_pipeline(StandardScaler(), KNeighborsClassifier())
search = GridSearchCV(pipe, {
    "kneighborsclassifier__n_neighbors": [3, 5, 9, 15, 25],
    "kneighborsclassifier__weights": ["uniform", "distance"],
    "kneighborsclassifier__p": [1, 2],
}, scoring="f1_macro", cv=5, n_jobs=-1)
search.fit(X_train, y_train)
print(search.best_params_)
print(classification_report(y_test, search.predict(X_test)))
```

8. Code Explanation

Wine measurements have different units, so scaling is essential and occurs inside folds. Grid search jointly selects neighborhood size, weighting, and Manhattan/Euclidean distance. Macro-F1 gives each wine class equal importance. The final test is used once after tuning.

9. Training / Evaluation

Tune (k), distance, and weights with CV. Plot validation score versus (k). Use stratified/grouped/time-aware splits as appropriate. For classification evaluate per-class recall/F1 and confusion; for regression use MAE/RMSE. Reduce irrelevant dimensions, encode categoricals thoughtfully, and handle missing values before distances.

10. Complexity and Cost

Brute-force "training" stores $(O(nd))$ data; one query costs $(O(nd))$ distance work plus neighbor selection. KD/Ball trees cost roughly $(O(nd \log n))$ to build and can approach $(O(\log n))$ query behavior in favorable low

dimensions, but degrade toward brute force in high dimensions. Inference memory/latency—not GPU training—is the concern.

11. Common Use Cases

Small-data image/handwriting classification, recommendation by similarity, local anomaly scores, missing-value imputation, pattern matching, and nonlinear tabular baselines.

12. Common Mistakes

Not scaling; including ID/noise features; choosing (k) on test data; using a random split for repeated users/time; ignoring class imbalance in voting; expecting fast serving with millions of stored points; treating one-hot Euclidean distance as automatically meaningful.

13. Edge Cases / Limitations

High dimensions make neighbors similarly distant. Dense regions dominate sparse ones under fixed (k). Duplicates at zero distance and equal-distance conflicting labels need defined behavior. Missing values break common metrics. Inference is slow and the full training data may create privacy/storage risk.

14. Variations

- **Radius Neighbors:** uses all samples within radius; handles variable density but can find none.
- **Distance-weighted KNN:** reduces influence of farther neighbors; common project option.
- **Approximate nearest neighbors:** trades exactness for scale; vector search/recommendation.
- **Edited/condensed KNN:** removes noisy/redundant prototypes; research/latency.
- **KNN regression:** neighbor-weighted continuous targets; placement relevant.

15. Related Topics

K-means learns prototypes rather than labels. Kernel regression is a smooth distance-weighted analogue. SVM RBF models local similarity with a learned sparse boundary. Approximate-neighbor vector databases extend the search idea to embeddings and RAG, but retrieval is not itself supervised KNN training.

16. Interview Questions

1. **Why is KNN nonparametric?** It assumes no fixed finite-dimensional functional form.
2. **Why lazy?** It stores examples; most computation happens at query time.
3. **Why scale?** Distance changes with numerical units.
4. **Small versus large (k)?** Small: flexible/noisy; large: smooth/biased.
5. **Training complexity?** Mostly storage/index construction; brute fitting is near ($O(nd)$).
6. **Inference complexity?** Brute force ($O(nd)$) per query.
7. **Why fail in high dimensions?** Sparse space and distance concentration make nearest neighbors less meaningful.
8. **KD tree versus brute?** KD trees help low dimensions; brute/vectorized search can win in high dimensions.
9. **How handle imbalance?** Class/distance-aware voting, resampling in folds, and suitable metrics.
10. **How choose a metric?** Based on feature semantics, scaling, sparsity, and validation—not habit.

17. Practice Tasks

Implement brute KNN with NumPy; visualize boundaries for several (k); add irrelevant dimensions and measure degradation; debug a missing scaler; compare exact and approximate search latency.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Similar-item recommender	Finds comparable products	sklearn/FAISS; retail catalog	Metric design and serving latency
Digit recognizer	Classifies handwritten digits	sklearn; MNIST subset	Boundary/error analysis
Local property estimator	Prices from nearby similar homes	pandas/sklearn; Ames	KNN regression and geospatial features

19. Quick Revision

Predict from (k) closest stored samples; scale and validate metric/(k); classification F1 or regression RMSE; traps: dimensions, latency, irrelevant features. **One-liner:** KNN exchanges almost-free training for local, data-heavy inference.

20. Final Cheat Sheet

Item	Summary
Input/output	Scaled query + stored examples → class/value
Main steps	distance → select neighbors → vote/average
Hyperparameters	(k), metric, (p), weights, search algorithm
Pros/cons	Simple nonlinear / slow inference, scale/dimension sensitive
Best use	Small low-dimensional datasets with local structure

Decision Tree

1. Overview

A decision tree recursively partitions feature space with if/then rules and predicts from the resulting leaves. Trees handle nonlinearities and interactions, require little numeric preprocessing, and are easy to visualize. They are used for tabular classification/regression, rule-based risk systems, and as the base learners of forests and boosting.

2. Intuition

It resembles the “20 Questions” game: ask the question that best separates outcomes, then repeat inside each answer group. A loan tree might ask `income <= 45k?`, then `late_payments <= 1?`; each leaf stores a class distribution or average target.

3. Prerequisites

Entropy/probability, variance, recursive partitioning, greedy algorithms, bias-variance, categorical encoding, classification/regression metrics.

4. Core Concepts

Concept	Meaning	Example/interview angle
Node/split	Rule ($x_j \leq t$) partitions samples	Greedily maximize impurity decrease
Leaf	Terminal region with prediction	Class proportions or mean response
Impurity	Heterogeneity of a node	Gini/entropy for class, MSE for regression
Depth	Longest root-to-leaf path	Main complexity/overfit control
Pruning	Remove weak branches	Pre-pruning params or cost-complexity pruning
Axis-aligned boundary	Each split tests one feature	Staircase approximation of diagonal curves

5. Algorithm / Working Process

Start with all training examples at the root. For candidate features and thresholds, compute weighted child impurity. Choose the largest decrease, split, and recurse. Stop at purity or constraints such as `max_depth`, `min_samples_leaf`, or minimum decrease. Optionally prune. A query traverses rules to one leaf; output its learned value or class probabilities.

6. Mathematical Foundation

For node (m) with class proportions (p_{mk}):

$$Gini(m) = 1 - \sum_k p_{mk}^2, \quad H(m) = - \sum_k p_{mk} \log_2 p_{mk}.$$

For split (s) producing left/right children:

$$\Delta I = I(\text{parent}) - \frac{n_L}{n} I(L) - \frac{n_R}{n} I(R).$$

Choose the split maximizing (ΔI). Regression commonly uses within-node squared error ($I(m) = \frac{1}{n_m} \sum_{i \in m} (y_i - \bar{y}_m)^2$). Cost-complexity pruning minimizes ($R(T) + \alpha |\widetilde{T}|$), balancing leaf error and number of leaves.

7. Practical Implementation

```

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import classification_report, roc_auc_score

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
tree = DecisionTreeClassifier(random_state=42, class_weight="balanced")
search = GridSearchCV(tree, {
    "max_depth": [3, 5, 8, None],
    "min_samples_leaf": [1, 5, 15, 30],
    "ccp_alpha": [0.0, 0.001, 0.01],
}, scoring="roc_auc", cv=5, n_jobs=-1)
search.fit(X_train, y_train)
prob = search.predict_proba(X_test)[:, 1]
print(search.best_params_)
print(classification_report(y_test, search.predict(X_test)))
print("ROC-AUC:", roc_auc_score(y_test, prob))
print(export_text(search.best_estimator_, max_depth=3))

```

8. Code Explanation

No standardization is needed because a monotonic scaling preserves possible order thresholds. Stratification preserves class ratios. Grid search regularizes through depth, minimum leaf population, and pruning. `export_text` makes the fitted rules auditable; truncated display does not alter the model.

9. Training / Evaluation

Trees can reach perfect training performance, so compare train/CV curves and tune structure. `min_samples_leaf` often stabilizes probabilities better than depth alone. Use class metrics and ROC/PR curves for classification; MAE/RMSE for regression. Avoid random splits for grouped/time data. Validate feature-importance conclusions with permutation importance on held-out data.

10. Complexity and Cost

Efficient balanced-tree training is commonly around ($O(dn \log n)$), though pathological behavior can be worse. Inference is ($O(h)$) comparisons per row, with height (h); a balanced tree has ($h \approx \log n$), a degenerate tree ($h \approx n$). Model memory is ($O(\text{ext}\{\text{nodes}\})$), up to ($O(n)$). CPU is sufficient.

11. Common Use Cases

Credit/risk rules, medical triage aids, customer segmentation with targets, interpretable tabular baselines, nonlinear regression, and base estimators in Random Forest/Extra Trees/boosting.

12. Common Mistakes

Growing an unrestricted tree; trusting training accuracy; using ordinal encoding whose category order creates misleading thresholds; treating impurity importance as causal; leaking target-derived features; reporting accuracy alone on imbalance; expecting smooth regression predictions.

13. Edge Cases / Limitations

Trees have high variance: small data changes can change early splits dramatically. Axis-aligned boundaries approximate diagonal/smooth functions inefficiently. Regression trees predict piecewise constants and do not extrapolate beyond leaf targets. Standard trees can favor high-cardinality split opportunities and produce poor raw probability estimates in tiny leaves.

14. Variations

- **CART:** binary classification/regression trees; dominant placement formulation.
- **ID3/C4.5/C5.0:** entropy/information-gain family with differing categorical/pruning behavior; interview history.
- **Oblique trees:** linear-combination splits; research/specialized projects.
- **Model trees:** fit models in leaves; smoother regression.
- **Conditional inference trees:** statistical split testing reduces selection bias; research.

15. Related Topics

Random Forest and Extra Trees reduce a single tree's variance by averaging randomized trees. Gradient boosting grows shallow trees sequentially to reduce errors. Rule lists prioritize compact interpretability. KNN also makes local nonlinear predictions but stores points instead of learned partitions.

16. Interview Questions

1. **Gini versus entropy?** Both measure impurity and often choose similar splits; Gini is slightly simpler computationally.
2. **Why no scaling?** Only relative order and thresholds matter for standard axis-aligned trees.
3. **Why do trees overfit?** Recursive splitting can isolate noise and tiny leaves.
4. **How prevent it?** Depth/leaf/split limits, impurity thresholds, pruning, or ensembles.
5. **Is split search global?** No, greedy local choices are not revisited.
6. **How does regression work?** Choose variance/MSE-reducing splits and predict leaf means (or medians for absolute-error criteria).
7. **What is pruning?** Removing branches whose complexity is not justified by validation/penalized risk.
8. **Why high variance?** Early split changes alter all descendants.
9. **Can it extrapolate?** Standard regression trees cannot extrapolate beyond learned leaf constants.
10. **Why biased feature importance?** Training impurity decreases can favor continuous/high-cardinality variables and overfit features.

17. Practice Tasks

Implement a one-level Gini split; plot depth versus train/CV score; compute a pruning path; debug ordinal category encoding; compare impurity and permutation importance after adding a random high-cardinality feature.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Loan rule explorer	Predicts approval/default and prints paths	sklearn; German Credit	Interpretability plus fairness checks
Triage tree	Creates auditable health-risk routing	sklearn; heart disease	Cost-sensitive error analysis
Quality-control tree	Flags manufacturing defects	pandas/sklearn; SECOM	Missing data and rule deployment

19. Quick Revision

Greedy impurity-reducing partitions; no scaling; tune depth/leaf/pruning; classification or regression metrics; traps: variance and misleading importance. **One-liner:** a decision tree converts feature space into axis-aligned regions with a simple prediction in each leaf.

20. Final Cheat Sheet

Item	Summary
Input/output	Tabular features → class/probability/value
Main steps	score splits → partition recursively → leaf prediction
Hyperparameters	criterion, depth, min samples, max features, <code>ccp_alpha</code>
Pros/cons	Interpretable nonlinear / unstable, overfits, stepwise
Best use	Auditable tabular rules and ensemble base learner

Random Forest

1. Overview

Random Forest averages many decorrelated decision trees. Each tree trains on a bootstrap sample and considers a random feature subset at each split. Averaging reduces variance while preserving nonlinear interactions, making Random Forest a robust tabular baseline for classification and regression.

2. Intuition

Instead of trusting one volatile expert, ask many experts trained on slightly different cases and limited clues. Their errors cancel when the experts are individually useful but not perfectly correlated.

3. Prerequisites

Decision trees, bootstrap sampling, bagging, bias-variance, feature subsampling, majority vote/averaging, out-of-bag evaluation.

4. Core Concepts

Concept	Meaning	Why/interview angle
Bootstrap	Sample (n) training rows with replacement per tree	About 63.2% unique rows on average
Feature randomness	Candidate subset at every node	Decorrelates trees
Aggregation	Vote/average across trees	Reduces variance, not usually bias
OOB samples	Rows omitted from a tree's bootstrap	Internal validation estimate
<code>n_estimators</code>	Number of trees	More stabilizes; diminishing returns, not classic overfit
<code>max_features</code>	Candidate features per split	Controls strength-correlation trade-off

5. Algorithm / Working Process

For each tree, draw a bootstrap sample; grow a usually deep tree, choosing every split from a random feature subset; repeat independently; at inference send the query through all trees; average class probabilities/values or vote. Optionally aggregate predictions for each training row only from trees where it was out-of-bag.

6. Mathematical Foundation

Regression aggregation is $\hat{f}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$; classification aggregates votes/probabilities. For identically distributed estimators with variance (σ^2) and pairwise correlation ρ ,

$$\text{Var}(\bar{T}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

Thus more trees reduce the independent component, while random feature selection targets correlation ρ . A bootstrap row is omitted with probability $((1-1/n)^n \rightarrow e^{-1} \approx 0.368)$, hence roughly 36.8% OOB rows per tree.

7. Practical Implementation

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, roc_auc_score

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
forest = RandomForestClassifier(
    n_estimators=500, oob_score=True, class_weight="balanced_subsample",
    random_state=42, n_jobs=-1
)
search = RandomizedSearchCV(forest, {
    "max_depth": [None, 5, 10, 20],
```

```
"min_samples_leaf": [1, 2, 5, 10],
"max_features": ["sqrt", "log2", 0.5, 1.0],
}, n_iter=15, scoring="roc_auc", cv=5, random_state=42, n_jobs=-1)
search.fit(X_train, y_train)
best = search.best_estimator_
prob = best.predict_proba(X_test)[:, 1]
print(search.best_params_, "OOB:", best.oob_score_)
print(classification_report(y_test, best.predict(X_test)))
print("ROC-AUC:", roc_auc_score(y_test, prob))
```

8. Code Explanation

Trees need no standard scaling. `balanced_subsample` recomputes class weights per bootstrap. OOB accuracy is a convenient development estimate, while CV is used for tuning and the test remains final. Parallel CPU cores build trees independently. Structural hyperparameters matter more than micro-tuning tree count after predictions stabilize.

9. Training / Evaluation

Start with many trees and sensible defaults; tune leaf size/depth and `max_features`. OOB is useful, but grouped/time-dependent data still requires appropriate external splits because ordinary bootstrapping violates structure. Use ROC/PR/F1 for classification and MAE/RMSE for regression. Check calibration if probabilities trigger decisions. Permutation importance should use held-out data and account for correlated-feature dilution.

10. Complexity and Cost

With (B) trees and candidate feature count (m) , a rough balanced cost is $(O(Bmn \log n))$, implementation/data dependent. Inference is $(O(Bh))$; memory is $(O(B \times \text{nodes}))$, often substantial for deep trees. Training parallelizes well across CPU; GPU is unnecessary for standard scikit-learn forests.

11. Common Use Cases

Credit/fraud/churn, medical risk, remote sensing, industrial faults, ecology, demand/price regression, tabular feature screening, and a dependable nonlinear baseline.

12. Common Mistakes

Believing more trees fix biased/poor features; using OOB with grouped/time data as if independent; trusting impurity importance; allowing huge forests to exhaust memory/latency; tuning on test; expecting extrapolation; using probability outputs without calibration checks.

13. Edge Cases / Limitations

Forests are less interpretable than one tree, large models are slow/heavy, and regression predictions remain averages of observed leaf targets rather than trends beyond the range. Very sparse high-dimensional linear text can favor logistic models. Correlated features split importance. Smooth functions may be approximated inefficiently with stepwise leaves.

14. Variations

- **Random Forest Regressor:** averages leaf values; placement essential.
- **Balanced Random Forest:** balanced bootstraps for severe imbalance; imbalanced-learn/project relevant.
- **Quantile Regression Forest:** estimates conditional quantiles/intervals; research and uncertainty projects.
- **Isolation Forest:** random isolation for anomaly detection; related mechanism, different objective.
- **Extremely randomized trees:** randomizes thresholds further; next chapter.

15. Related Topics

Bagging uses bootstraps but may consider all features; Random Forest adds feature randomness. Extra Trees adds random thresholds and often uses the full sample. Gradient boosting trains trees sequentially to reduce bias, whereas a forest trains them independently to reduce variance. XGBoost often wins tuned tabular benchmarks but requires more careful tuning.

16. Interview Questions

1. **Why better than one tree?** Averaging decorrelated high-variance trees reduces variance.
2. **What creates randomness?** Bootstrap rows and random candidate features at each split.
3. **Why feature subsampling?** Prevents dominant predictors from making all trees similar.
4. **What is OOB?** Per-row prediction from trees whose bootstrap omitted that row.
5. **Why 63.2% unique samples?** Probability of omission tends to (e^{-1}) , so inclusion is $(1 - e^{-1})$.
6. **Can more trees overfit?** Test error generally stabilizes; cost grows, while biased trees remain biased.
7. **Most useful hyperparameters?** Leaf size/depth, `max_features`, sampling/weights, tree count for stability.
8. **Does scaling matter?** Not for ordinary threshold splits.
9. **Random Forest versus boosting?** Parallel variance reduction versus sequential bias/error correction.
10. **How get importance reliably?** Held-out permutation importance/SHAP with correlation and leakage caveats.

17. Practice Tasks

Derive/verify 63.2% bootstrap uniqueness; plot OOB error versus tree count; compare one tree/forest; inject a random high-cardinality feature and audit importance; reduce model memory through leaf constraints.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Fraud ranker	Scores transactions under imbalance	sklearn; IEEE-CIS sample	PR metrics, calibration, importance
Forest-cover classifier	Predicts land-cover class	sklearn; UCI Covertypes	Scale, multiclass, performance engineering
Predictive maintenance	Estimates equipment failure risk	sklearn/MLflow; NASA turbofan-derived labels	Group/time validation and monitoring

19. Quick Revision

Bootstrap + random features + tree averaging; no scaling; tune leaf/depth/features; OOB is convenient; traps: memory and importance bias. **One-liner:** Random Forest reduces tree variance by averaging strong, deliberately decorrelated trees.

20. Final Cheat Sheet

Item	Summary
Input/output	Tabular features → class probability/value
Main steps	bootstrap → random-feature trees → aggregate
Hyperparameters	trees, max features, depth, min leaf, sampling/weights
Metrics	task metrics plus OOB development estimate
Pros/cons	Robust nonlinear baseline / large, less interpretable, no extrapolation
Best use	General tabular classification/regression

Extra Trees

1. Overview

Extra Trees (Extremely Randomized Trees) is an ensemble of randomized decision trees. Compared with a conventional Random Forest, it randomizes split thresholds more strongly and, by default in scikit-learn, trains each tree on the entire training sample rather than a bootstrap. This often reduces variance and training cost at the price of somewhat higher bias.

2. Intuition

Random Forest lets each expert search carefully for the best question among a random set of topics. Extra Trees also proposes random question cutoffs, then chooses the best among those proposals. Less optimization per tree creates more diverse experts whose average can generalize well.

3. Prerequisites

Decision trees, Random Forest, bagging, random feature subsets, bias-variance, ensemble aggregation, classification/regression evaluation.

4. Core Concepts

Concept	Meaning	Why/interview angle
Random thresholds	Candidate cut points are sampled rather than exhaustively optimized	More diversity, faster split search

Concept	Meaning	Why/interview angle
Whole sample default	sklearn uses <code>bootstrap=False</code>	Key difference from Random Forest defaults
Feature randomness	Only a subset is considered per split	Decorrelates trees further
Aggregation	Average probabilities/values	Reduces variance
Bias-variance trade	Extra randomness can add bias, lower variance	Which wins is data-dependent

5. Algorithm / Working Process

For every tree, use the full sample by default (or bootstrap if explicitly enabled). At each node select random candidate features, generate random split thresholds for them, evaluate the randomized candidates, and choose the best. Grow under structural constraints. Aggregate all trees by probability/value averaging. Inference otherwise resembles Random Forest.

6. Mathematical Foundation

The ensemble predictor is

$$\hat{f}(x) = \frac{1}{B} \sum_{b=1}^B T(x; \mathcal{D}, \theta_b),$$

where (θ_b) captures random feature and cut-point choices. Each candidate split still uses impurity decrease

$$\Delta I = I(\text{parent}) - \frac{n_L}{n} I(L) - \frac{n_R}{n} I(R),$$

but it searches a randomized subset of possible thresholds. The same correlated-estimator variance expression $(\rho\sigma^2 + (1-\rho)\sigma^2/B)$ explains why extra decorrelation can help, while excessive randomization can weaken individual trees.

7. Practical Implementation

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.metrics import classification_report, roc_auc_score

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)
model = ExtraTreesClassifier(
    n_estimators=500, class_weight="balanced",
    random_state=42, n_jobs=-1
```

```

)
search = GridSearchCV(model, {
    "max_features": ["sqrt", 0.5, 1.0],
    "min_samples_leaf": [1, 2, 5, 10],
    "max_depth": [None, 8, 16],
}, scoring="f1_macro", cv=5, n_jobs=-1)
search.fit(X_train, y_train)
prob = search.predict_proba(X_test)
print(search.best_params_)
print(classification_report(y_test, search.predict(X_test)))
print("Multiclass ROC-AUC:", roc_auc_score(y_test, prob,
                                          multi_class="ovr"))

```

8. Code Explanation

No scaler is required for threshold trees. A fixed seed makes randomized fitting reproducible. The search controls individual-tree strength and regularization; 500 trees stabilizes averaging. Macro-F1 prevents a common class from hiding weak minority-class results. Bootstrap remains off, preserving the defining sklearn default difference.

9. Training / Evaluation

Evaluate Extra Trees and Random Forest on identical splits rather than assuming one wins. Tune `max_features`, leaf size, depth, and tree count. Use task metrics plus latency/memory. OOB scoring is unavailable unless bootstrapping is enabled. Check calibration and held-out permutation importance. Use grouped/time-aware validation where needed.

10. Complexity and Cost

Rough training cost is $(O(Bmn \log n))$ for balanced trees, but random threshold generation can reduce split-search constants versus optimizing all thresholds. Inference is $(O(Bh))$; memory is $(O(B \times \text{nodes}))$. Trees parallelize across CPU cores. Fully deep ensembles can still be large.

11. Common Use Cases

Fast nonlinear tabular baselines, high-dimensional feature screening, remote sensing, biomedical classification, anomaly-related feature embeddings, and regression where Random Forest variance is high.

12. Common Mistakes

Saying Extra Trees always bootstraps; saying splits are selected with no impurity evaluation at all; treating it as identical to Random Forest; relying on training impurity importance; assuming randomization always improves accuracy; using too few trees; expecting regression extrapolation.

13. Edge Cases / Limitations

Random cutoffs can add too much bias on simple small datasets. Like forests, it is memory-heavy, less interpretable, stepwise, and weak at extrapolation. Tiny leaves can give poor probabilities. Very sparse linear text problems may favor linear models. Default no-bootstrap means ordinary OOB estimates are not available.

14. Variations

- **ExtraTreesRegressor:** average randomized regression trees; placement relevant.
- **Bootstrap Extra Trees:** enable row bootstrapping and OOB scoring; test empirically.
- **Random Trees Embedding:** unsupervised randomized leaf indicators for downstream models.
- **Totally randomized trees:** even less split optimization; research/representation learning.

15. Related Topics

Random Forest usually bootstraps rows and optimizes thresholds more thoroughly. Bagging mainly randomizes samples. Random Trees Embedding converts leaf membership to sparse features. Gradient boosting uses carefully optimized sequential trees and targets residual errors, unlike independent Extra Trees.

16. Interview Questions

1. **What does “extra” mean?** Extra randomization, especially random split thresholds.
2. **Extra Trees versus Random Forest?** More randomized thresholds; sklearn Extra Trees defaults to the full sample, RF to bootstrap samples.
3. **Are thresholds completely accepted at random?** Random candidates are generated, then the best candidate by criterion is chosen.
4. **Why can it train faster?** It avoids exhaustive threshold optimization at nodes.
5. **Bias-variance effect?** Usually higher bias and lower variance/correlation.
6. **Does it require scaling?** Not for standard axis-aligned thresholds.
7. **Can it use OOB scoring?** Only with bootstrapping enabled in sklearn.
8. **When can it beat RF?** When variance reduction/diversity outweighs weaker individual splits.
9. **How tune it?** Trees for stability, then feature fraction, leaf size, depth, and optional bootstrap.
10. **Main interpretability issue?** Hundreds of randomized trees cannot be summarized as one trustworthy rule set.

17. Practice Tasks

Compare RF/Extra Trees over repeated seeds; toggle bootstrap and inspect OOB; plot latency versus number of trees; debug the claim that thresholds are never scored; compare permutation and impurity importance under correlated features.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Land-cover benchmark	Compares tree ensembles	sklearn; Covertypes	Controlled benchmarking and scale
Defect classifier	Detects rare manufacturing faults	sklearn; SECOM	Imbalance, missing values, error costs
Property valuation	Nonlinear price prediction	sklearn; Ames Housing	RF/Extra Trees ablation and serving

19. Quick Revision

Random features + random thresholds + ensemble averaging; usually no bootstrap by default; tune leaves/features/trees; traps: RF confusion and biased importance. **One-liner:** Extra Trees trades more split randomness for stronger decorrelation and often faster fitting.

20. Final Cheat Sheet

Item	Summary
Input/output	Tabular features → class probability/value
Main steps	randomized split candidates → many trees → average
Hyperparameters	trees, max features, min leaf, depth, bootstrap
Metrics	Standard task metrics, calibration, latency/memory
Pros/cons	Fast, diverse, strong baseline / extra bias, large model
Best use	Variance-heavy nonlinear tabular problems

Support Vector Machine

1. Overview

A Support Vector Machine (SVM) finds a decision boundary with maximum margin between classes. Soft margins tolerate violations, and kernels create nonlinear boundaries through implicit feature mappings. SVMs are effective on medium-size, high-dimensional datasets such as text, bioinformatics, and image descriptors. Support Vector Regression (SVR) extends the principle to continuous targets.

2. Intuition

Many lines may separate two classes; choose the one with the widest empty street between them. Only the points touching or violating the street—the support vectors—determine it. A kernel measures similarity as though points had been mapped to a richer space without explicitly constructing that map.

3. Prerequisites

Vectors/dot products/norms, hyperplanes, constrained convex optimization, Lagrange multipliers and duality at interview depth, scaling, kernels, classification metrics.

4. Core Concepts

Concept	Meaning	Why/interview angle
Hyperplane	$(w^T x + b = 0)$	Linear decision boundary
Margin	Distance to closest classes; canonical width $(2/ w)$	Maximizing it controls capacity
Support vectors	Points with nonzero dual coefficients	Determine the boundary/inference cost

Concept	Meaning	Why/interview angle
(C)	Penalty for margin violations	High (C): fit training harder; low (C): wider margin
Kernel	$K(x,z)=\phi(x)^T\phi(z)$	Implicit nonlinear feature space
γ	RBF influence scale	High γ gives highly local/wiggly boundaries

5. Algorithm / Working Process

Scale features; encode binary labels as $-1/+1$; choose linear or kernel form; solve the convex soft-margin problem; retain support vectors and coefficients. At inference, a linear SVM evaluates (w^Tx+b) ; a kernel SVM sums weighted kernel similarities to support vectors. The sign/argmax yields a class; probabilities require separate calibration.

6. Mathematical Foundation

Hard-margin primal:

$$\min_{w,b} \frac{1}{2} \|w\|_2^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1.$$

Soft margin introduces slack ξ :

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i, \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Equivalently minimize regularized hinge loss ($\max(0, 1 - y_i f(x_i))$). The dual prediction is

$$f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b.$$

RBF kernel: $K(x,z)=\exp(-\gamma|x-z|^2)$. Linear/polynomial/RBF kernels must satisfy positive-semidefinite conditions for the standard convex interpretation. SVR uses an ϵ -insensitive tube: errors inside $\pm\epsilon$ receive no loss.

7. Practical Implementation

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, roc_auc_score

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
pipe = make_pipeline(StandardScaler(), SVC(class_weight="balanced"))
```

```

search = GridSearchCV(pipe, [
    {"svc__kernel": ["linear"], "svc__C": [0.01, 0.1, 1, 10]},
    {"svc__kernel": ["rbf"], "svc__C": [0.1, 1, 10, 100],
     "svc__gamma": ["scale", 0.001, 0.01, 0.1]},
], scoring="roc_auc", cv=5, n_jobs=-1)
search.fit(X_train, y_train)
score = search.decision_function(X_test)
pred = search.predict(X_test)
print(search.best_params_)
print(classification_report(y_test, pred))
print("ROC-AUC:", roc_auc_score(y_test, score))

```

8. Code Explanation

Scaling is inside CV because distance, dot products, (C), and γ depend on feature units. Separate parameter grids avoid searching meaningless γ values for a linear kernel. `decision_function` is enough for threshold-free ROC-AUC; `probability=True` is omitted because it adds calibration cost. Class weights adjust margin-violation costs.

9. Training / Evaluation

Try a linear baseline first for large/sparse/high-dimensional data. Tune (C) and γ logarithmically and jointly for RBF. High (C)+high γ often overfits; low values can underfit. Use stratified/nested CV, appropriate class metrics, and learning curves. If probabilities matter, calibrate on held-out folds and evaluate log loss/Brier/calibration, not just AUC.

10. Complexity and Cost

Kernel SVM training commonly scales between ($O(n^2d)$) and ($O(n^3)$) in difficult cases and needs roughly ($O(n^2)$) kernel memory, so it struggles with very large (n). Inference costs ($O(sd)$) kernel work per row for (s) support vectors. Linear SVMs with primal/SGD-style solvers can approach ($O(knd)$) and scale far better. CPU is typical; specialized GPU libraries exist but do not remove algorithmic scaling.

11. Common Use Cases

Text/document classification, gene-expression/biomedical data, handwriting/image descriptors, medium-size nonlinear tabular classification, novelty detection (`OneClassSVM`), and nonlinear regression (`SVR`).

12. Common Mistakes

Not scaling; using RBF SVC blindly on millions of rows; interpreting decision scores as probabilities; enabling probability calibration unnecessarily; tuning (C, γ) on test data; searching γ linearly; confusing high (C) with stronger regularization—it means weaker regularization/more violation cost.

13. Edge Cases / Limitations

Kernel training is expensive at large (n), multiclass generally requires several binary problems, predictions slow when most rows become support vectors, and kernel explanations are difficult. Overlapping/noisy classes

need soft margins. Poor scaling or irrelevant dimensions distort RBF distances. SVM probability estimates are an extra calibrated layer, not intrinsic maximum-margin outputs.

14. Variations

- **LinearSVC:** scalable linear classifier using liblinear; large sparse placement/project data.
- **NuSVC:** uses ν to control support-vector/error bounds; advanced interview topic.
- **SVR/LinearSVR:** ϵ -insensitive regression; placement relevant.
- **One-Class SVM:** novelty/anomaly boundary; specialized projects.
- **Kernel approximations:** random Fourier/Nyström features plus linear model; scalable nonlinear approximation.

15. Related Topics

Logistic regression uses smooth probabilistic log loss; linear SVM uses hinge loss/margin and usually no native probability. Kernel Ridge uses squared loss with kernels. KNN also depends on geometry but makes local votes rather than learning a maximum-margin boundary. Neural networks learn the representation instead of choosing a fixed kernel.

16. Interview Questions

1. **What is the margin?** Distance between class-supporting hyperplanes; canonical total width is $(2/|w|)$.
2. **Why maximize it?** A wider margin controls model capacity and can improve generalization.
3. **What are support vectors?** Training points with nonzero dual influence on the boundary.
4. **What does high (C) do?** Penalizes violations strongly, fits training more closely, effectively less regularization.
5. **What does high γ do in RBF?** Makes influence very local and the boundary more complex.
6. **Why scale?** Margin and kernels are functions of coordinates/distances.
7. **What is the kernel trick?** Compute inner products in an implicit feature space using $(K(x,z))$.
8. **SVM versus logistic?** Hinge/max-margin versus log-likelihood/probabilities.
9. **Why not kernel SVM for huge data?** Quadratic memory and superlinear training plus support-vector inference cost.
10. **How multiclass works?** Common implementations combine binary models, e.g. one-vs-one; linear alternatives may use one-vs-rest.

17. Practice Tasks

Implement hinge loss; visualize (C/γ) boundary changes; compare **LinearSVC** and RBF **SVC** as (n) grows; debug an unscaled model; calibrate decision scores and compare Brier score.

18. Project Ideas

Project	What it does	Stack/dataset	Resume value
Support-ticket router	Linear SVM on TF-IDF	sklearn/FastAPI; public issue text	Sparse scaling and deployment
Lesion-feature classifier	RBF SVM on extracted descriptors	sklearn/OpenCV; skin-lesion metadata/features	Kernel tuning and medical metrics

Project	What it does	Stack/dataset	Resume value
Predictive SVR	Predicts equipment remaining life	sklearn; NASA turbofan	ϵ -insensitive regression and group splits

19. Quick Revision

Maximum-margin classifier; soft margin uses (C), RBF locality uses γ ; always scale; use decision scores unless calibrated probabilities are required; trap: kernel scaling with sample count. **One-liner:** SVM bases a maximum-margin boundary on the hardest, most informative training points.

20. Final Cheat Sheet

Item	Summary
Input/output	Scaled features → decision score/class (or SVR value)
Main steps	choose kernel → solve margin objective → retain support vectors
Hyperparameters	(C), kernel, γ , degree, class weights; SVR ϵ
Metrics	F1/ROC-AUC/PR-AUC; RMSE/MAE for SVR; calibration if needed
Pros/cons	Strong medium/high-d data / scaling, costly kernels, opaque probabilities
Best use	Medium-size high-dimensional or nonlinear problems

Cross-Algorithm Selection Map

Situation	Start with	Why
Continuous target, interpretation required	Linear/Ridge/Lasso/Elastic Net	Transparent coefficients and strong diagnostics
Smooth low-dimensional curve	Polynomial Ridge	Controlled nonlinear basis
Binary probability baseline	Logistic Regression	Efficient, regularized, probability-oriented objective
Sparse text and little labeled data	Multinomial/Complement NB	Very fast and data-efficient
Small, low-dimensional, irregular boundary	KNN	Flexible local behavior
One auditable rule system	Pruned Decision Tree	Human-readable paths
General nonlinear tabular baseline	Random Forest or Extra Trees	Low-preprocessing, robust ensembles
Medium-size high-dimensional boundary	Linear/RBF SVM	Strong margin-based generalization

Authoritative References

- [scikit-learn: Linear models](#)
- [scikit-learn: Naive Bayes](#)
- [scikit-learn: Nearest Neighbors](#)
- [scikit-learn: Decision Trees](#)
- [scikit-learn: Ensemble methods](#)
- [scikit-learn: Support Vector Machines](#)
- [Breiman \(2001\), "Random Forests"](#)
- [Geurts, Ernst, and Wehenkel \(2006\), "Extremely randomized trees"](#)